

MASTER'S THESIS

E-graphs modulo theories
with applications to AC

Sofia Brookie

sofia.ma@protonmail.com

Supervisor: Nicholas Smallbone

Examiner: David Sands

May 26, 2026

Contents

1	Abstract	3
2	Introduction to E-graphs	5
2.1	Equational theorem proving	5
2.2	Union-find, congruence closure, and equality saturation	6
2.2.1	E-graphs for congruence-closure	8
2.2.2	Aside: variations on equality saturation	17
3	Egraphs modulo AC	18
3.1	Handling AC with EMTs: a first example	20
3.2	Challenges for EMT(AC)	22
3.3	Filling the gap	25
3.4	Prospects for EMT(AC)	25
4	E-graphs, EMTs, and rewriting	27
4.1	E-graphs present a rewriting system	27
4.2	E-graphs simplify confluence	28
4.3	Equality satuation: handling AC-critical pairs by brute force	29
4.4	EMTs: Handling AC-critical pairs less naïvely	30
4.4.1	EMT(AC)s via a side-table	30
4.4.2	Integrated EMT(AC)s	31
5	Beyond rewriting: E-matching and E-analysis	32
6	Implementation and evaluation	33
6.1	Laziness and incrementality	33
6.1.1	Union-find laziness	33
6.2	Evaluation	33
7	EMTs for other theories	34
8	Prior work	35

Chapter 1

Abstract

E-graphs are a data structure for equational theorem proving, and which provide the basis for *equality saturation* (section **FiXme: TODO**), a method to automatically derive equational consequences from a starting set of terms, equations, and identities. E-graphs have generated a lot of interest since the publication of the EGG library developed by Willsey et al. and their corresponding paper ([8]).

However, E-graphs struggle with a combinatorial explosion when reasoning about certain theories. A notorious case is that of AC theories, those containing associative and commutative operators: the space complexity of saturation is doubly-exponential in the size of the input in the worst case, as shown in section **3**.

Can equality saturation for AC theories be done in a more efficient way? There have been several proposals for extending E-graphs to *E-graphs modulo theories* (EMTs) ([9], [3], [4]). These proposals suggest integrating a *theory-specific* reasoning method for AC together with the general mechanisms of E-graphs and equality saturation in order to efficiently deal with AC theories. The hope is that specific methods for AC theories can avoid the inefficiencies in using the general equality saturation mechanism for the AC identities.

EMTs are inspired by the success of theory-specific methods in other automated reasoning tasks, such as in *Satisfiability Modulo Theories* solvers (SMT solvers), and the merits of a theory-specific approach in automated reasoning are advocated for in Shankar [5]

We will develop the idea of EMTs and show that it *is* possible to reason with AC theories and a variety of closely related theories in a way that avoids the worst inefficiencies of plain E-graphs.

In section **Fixme: TODO**, we also provide a formal definition of EMTs that we hope can provide the basis for future work on integrating other theories in a similar manner.

We then, in section 6, discuss an implementation of EMTs and benchmark it against pure E-graph reasoning on a variety of equational reasoning tasks (6.2).

Central to our notion of a ‘good’ theory for an EMT is the necessity of a solver for the word problem over the theory, which we justify in section **Fixme: TODO**. This gives us a lower bound on EMTs for AC: the word problem for AC requires in the worst case exponential space and doubly-exponential time. This is an improvement over the doubly-exponential space upper bound for E-graphs, but highlights that concrete efficiency gains derive from the potential for typical problems to be solvable efficiently. Our solver for AC and polynomials is built upon Buchberger’s algorithm, which has received significant effort to achieve good common-case performance.

On the other hand, for the theory ACUN (AC with unit and negation, aka the theory of Abelian groups) or the theory of vector spaces, we have polynomial time bounds: we can use algorithms for Hermite normal form (for Abelian groups) and Gaussian elimination (for vector spaces).

Chapter 2

Introduction to E-graphs

2.1 Equational theorem proving

The problem we are interested in solving is to take a set of equations and figure out the set of consequences of those equations. There are some different cases to consider:

1. We know the goal equation, so we want to either *validate* that it follows from the input equations or determine that it doesn't follow
2. We don't know the goal equation, and we want to explore some interesting equations that follow from the input equations

E-graphs can be used for both, but they particular shine in the second case, as they are not fundamentally 'goal-directed'.

Let's consider an example.

$$f(x, y) = f(y, x) \tag{2.1}$$

$$f(1, 1) = 2 \tag{2.2}$$

$$f(2, 1) = 3 \tag{2.3}$$

Here, the first equation is understood to hold for any values of x and y . We will call such an equation an *identity*: really, it stands for the universally quantified formula $\forall xy. f(x, y) = f(y, x)$. We will then reserve the term 'equation' for *ground* equations: one without such quantifiers.

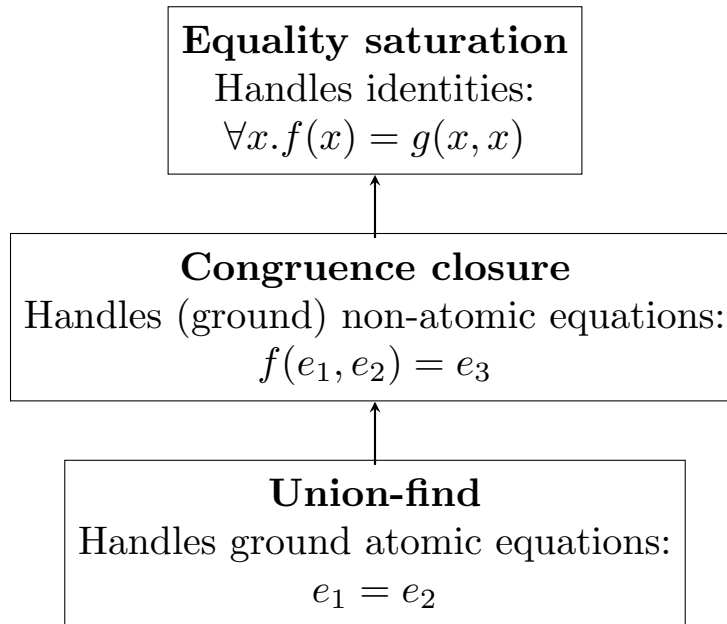
Now, here is one consequence of the above equations:

$$\begin{aligned}
f(1, f(1, 1)) & \\
&= f(f(1, 1), 1) && \text{by 2.1} \\
&= f(2, 1) && \text{by 2.2} \\
&= 3 && \text{by 2.3}
\end{aligned}$$

E-graphs are a data structure on which we can run *equality saturation* to discover the above equational consequence automatically, alongside other consequences.

Applications of equality saturation include optimization in compilers, decompilation, and program synthesis ([8]) **FiXme: need ref for program synthesis. Also theorem proving?**.

2.2 Union-find, congruence closure, and equality saturation



Describing E-graphs can be done in 3 layers. First, there is the union-find structure, which is a simple solver for the theory of atomic equations: given a set of *atoms* $e_1, e_2, \dots$, we can union two of them to set them equal, and we can find a representative of any particular atom. The representative is unique, so we can simply compare representatives for equality to determine if the atoms they represent are equivalent.

The data structure for this can be thought of as presenting a rewriting system among atoms: for instance,

$$\begin{aligned} e_1 &\rightarrow e_1 \\ e_2 &\rightarrow e_2 \\ e_3 &\rightarrow e_3 \\ e_4 &\rightarrow e_4 \end{aligned}$$

is the trivial equivalence relation on 4 atoms. Unioning e_1 and e_2 gives

$$\begin{aligned} e_1 &\rightarrow \underline{e_2} \\ e_2 &\rightarrow e_2 \\ e_3 &\rightarrow e_3 \\ e_4 &\rightarrow e_4 \end{aligned}$$

where we have (arbitrarily) chosen e_2 as the new representative.

If we union e_3 with e_4 , we get

$$\begin{aligned} e_1 &\rightarrow e_2 \\ e_2 &\rightarrow e_2 \\ e_3 &\rightarrow \underline{e_4} \\ e_4 &\rightarrow e_4 \end{aligned}$$

Now, if we union e_3 and e_2 , we could get

$$\begin{aligned} e_1 &\rightarrow e_2 \\ e_2 &\rightarrow \underline{e_4} \\ e_3 &\rightarrow e_4 \\ e_4 &\rightarrow e_4 \end{aligned}$$

Note that we do a lookup before performing the union, so that we union the two *representatives* (e_4 of e_3 and e_2 of e_2) together.

Now it takes two steps to rewrite e_1 to e_4 . It is possible to either eagerly canonize the rewrite system on update, setting e_1 to rewrite to e_4 , or do it lazily (compressing such multi-step rewrites on lookup), which is asymptotically more efficient in general. However, these efficiency gains are hard to realize given the congruence closure structure we will build on top of the union-find structure.

On top of union-find, we can build a congruence-closure structure. To handle non-atomic terms, we have to consider function symbols applied to atoms and other terms (like $f(g(e_1), e_2)$), and we need to respect *congruence*: if $s_i = t_i$, then we must conclude $f(s_1, \dots) = f(t_1, \dots)$.

To handle congruence, we will break terms down into *layers*. For instance, to represent $f(g(e_1), e_2)$ we will assign a new constant e_3 for $g(e_1)$, and then assign a constant e_4 for $f(e_3, e_2)$. This way, all congruence checks only need to apply to ‘flat’ terms: if $e_i = d_i$ then we conclude $f(e_1, \dots) = f(d_1, \dots)$.

This motivates the E-graph data structure, and the congruence closure algorithm that restores the congruence invariant. Computing the congruence closure is also known as ‘rebuilding’ the E-graph.

2.2.1 E-graphs for congruence-closure

As an example, we will compute the congruence closure of the following set of equations:

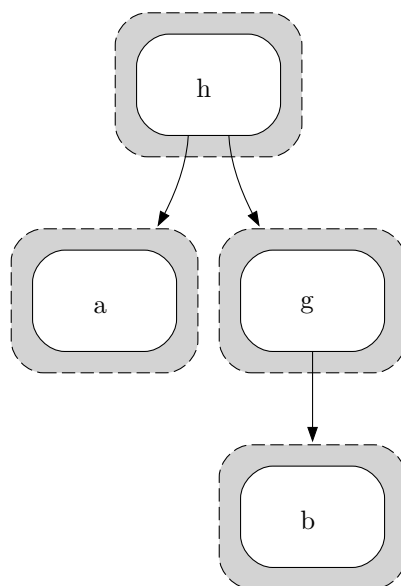
$$f(g(a), c) = h(a, g(b)) \tag{2.4}$$

$$h(g(c), b) = h(a, g(c)) \tag{2.5}$$

$$g(b) = g(c) \tag{2.6}$$

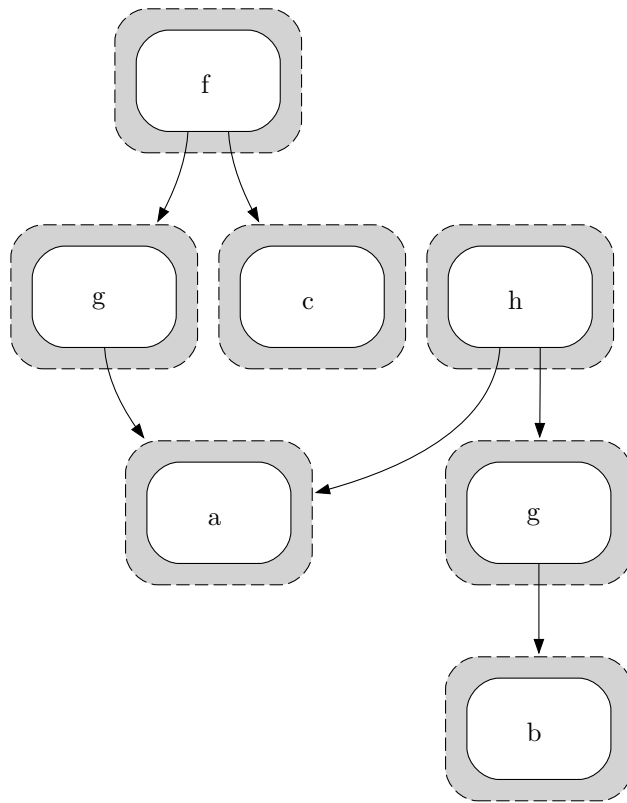
$$f(g(a), c) = g(b) \tag{2.7}$$

As a first step, we represent the term $h(a, g(b))$ from 2.4.

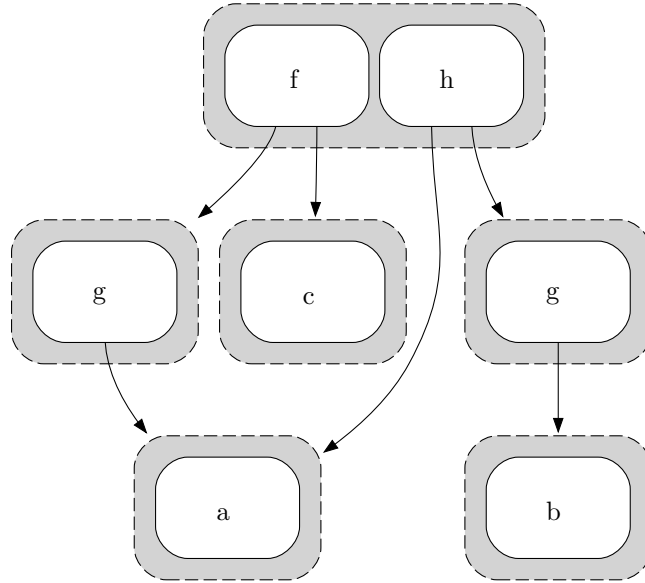


Notice how we have flattened the term: the three levels of nesting become pointers between the three layers of our graph.

Then we will add the other term $f(g(a), c)$ from [2.4](#).

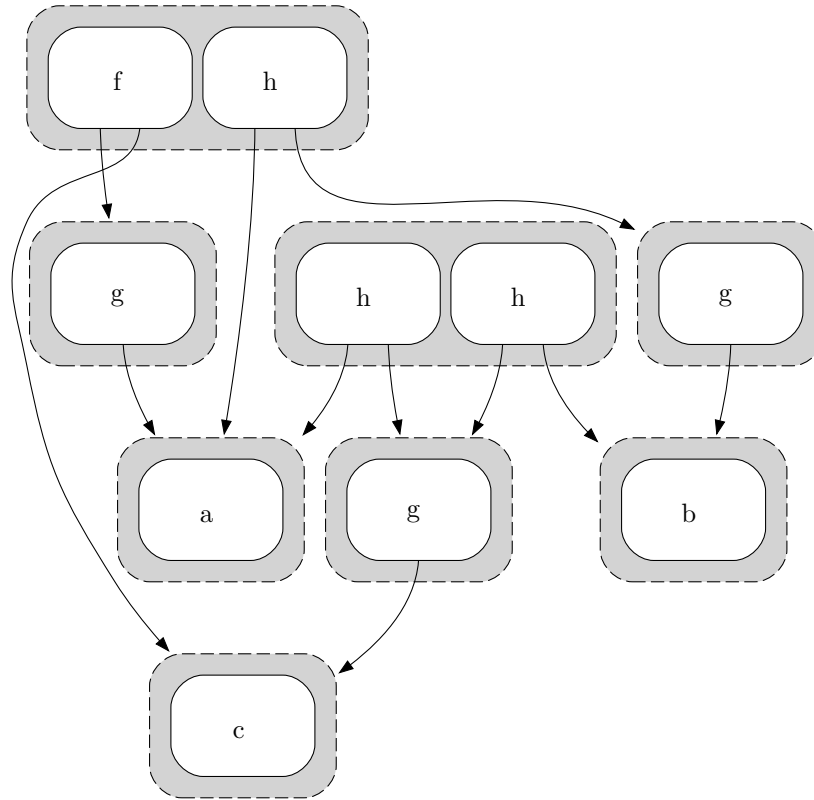


Now, as a final step, we need to *union* the two top nodes of both sides of the equation:

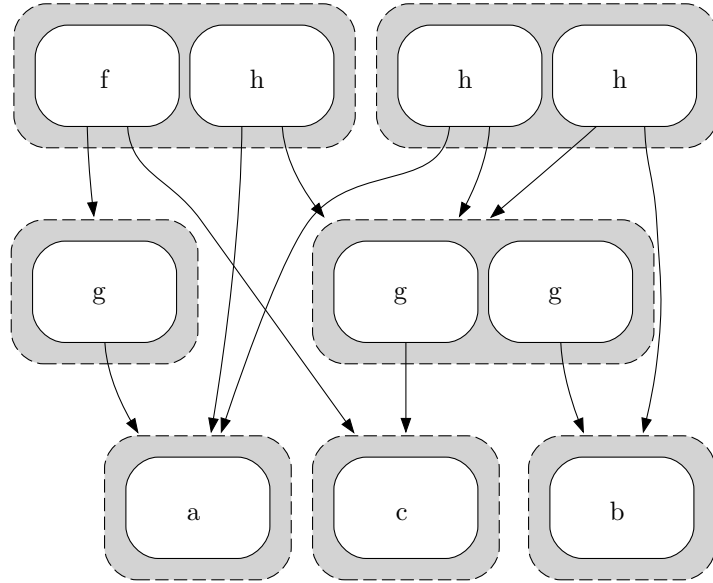


We call the outer nodes *E-classes* (equivalence classes), while the inner nodes are *E-nodes*. E-classes can contain multiple E-nodes. The arrows point from E-nodes to E-classes.

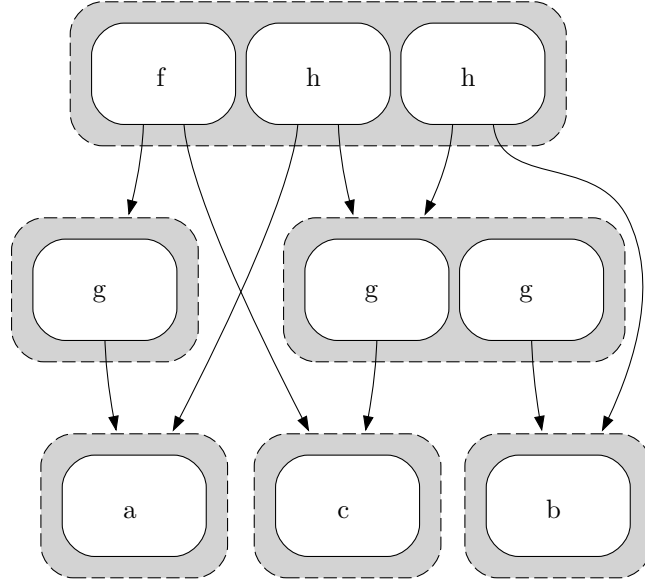
Next we move on to 2.5. This time, notice that the subterm $g(c)$ is used twice, and we *share* this in the E-graph, indicated by two arrows pointing to the E-class for $g(c)$.



Now we handle 2.6, which doesn't add any new E-nodes in the graph. Instead, it merges the separate E-classes representing $g(b)$ and $g(c)$ into a single E-node.

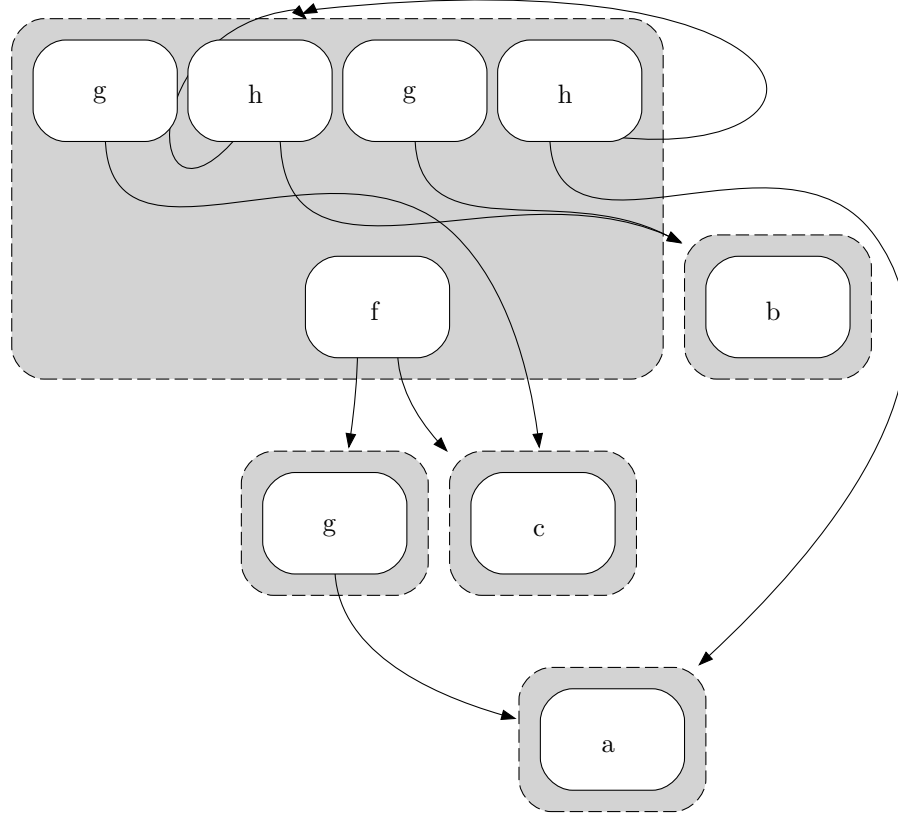


Now notice that we have two instances of an h with the same arguments: $h(a, g(b))$ and $h(a, g(c))$ have become equal by congruence, but they are located in two different E-classes. So, after unioning, we need to *propagate* by continuing to union things that have become equal. This gives us the simpler E-graph



Now we handle [2.7](#). This time, it will create a *cycle* in the E-graph: an E-node pointing to its own E-class. A simple equation that generates a cycle is $g(d) = d$, which gives rise to the infinite set of equations $g^n(d) = d$ by congruence.

In general, we might have cycles going through multiple intermediary nodes: $f(\dots g(\dots e_1 \dots) \dots) = e_1$. Cycles become very important in the analysis of how theories fare with E-graphs under equality saturation (section [FiXme: TODO](#)).



Handling congruence closure in E-graphs with cycles is not difficult: we can just propagate until nothing changes. There is a finite upper bound on propagation: at most, we could discover that every E-node is equal to every other. Every step moves closer to that bound by merging some E-nodes. Thus, this fix-point algorithm clearly terminates.

Equality saturation

Suppose instead of the equations $h(g(c), b) = h(a, g(c))$ in the above example we had the identity $\forall x. h(a, x) = h(x, b)$. To do equality saturation with this identity, we would *match* the left-hand-side $h(a, x)$ against our E-graph, which would find the match $h(a, g(b))$, corresponding to the substitution $x = g(b)$. Then, we apply the same substitution to the right hand side, thus finding the equations $h(a, g(b)) = h(g(b), b)$. Inserting this equation into our E-graph and using congruence closure would then yield the above E-graph.

Equalities are symmetric, so we need to match the right-hand-side and add the substituted left-hand-side in general equality saturation.

So, in general, to handle identities, we use the following procedure:

1. Add every ground equation to the E-graph
2. Equality saturation outer loop:
 - (a) (E-match) Match any side of an identity to a ground term represented by the E-graph
 - (b) (Insert) the other side: using the substitution from the previous step, construct the term resulting from applying the substitution to the other side of the identity. If it is not already represented by the E-graph, add it to the E-graph.
 - (c) (Merge) the E-classes representing the two sides from above
 - (d) Repeat

If the fixed point is reached, we call the graph **saturated**.

In practical applications such as compilers, we often have a final step to *extract* some minimal term equal to the starting term. Extraction is covered later
 FiXme: TODO

Equality saturation: limitations

When both substituted terms from E-matching an identity are already represented by the graph, we call the merge a *proper merge*. Unfortunately, proper merges aren't enough. If we consider the associative identity $\forall xyz. x + (y + z) = (x + y) + z$, and we begin with the two terms $w + (x + (y + z))$ and $((w + x) + y) + z$, we would like to prove that these terms are equal. However, the attempt to prove even the first step, that $w + (x + (y + z)) = (w + x) + (y + z)$, founders. Our representation includes no equivalence class to represent $(w + x) + (y + z)$, and so there are no equivalence classes to merge.

The 'solution' is to include the possibility of **inserting** new classes during the procedure. We call a mix of insertion and a merge an **improper merge**.

Insertion has one key problem: we now have no guarantees the process of merging will terminate. With only proper merges, there are a finite set of input terms and thus a finite bound on the equivalence relation we maintain. Thus, eventually we will get to a point where no merge changes the equivalence relation, and we have reached saturation. Introducing new classes for inserted terms now means that saturation is no longer guaranteed to be reached at any point, and worse, the strategy for insertion and merging now can affect whether

or not saturation is reached.

To even get a *semi-decision procedure*, where we aim to prove a specific equality if it exists and can fail otherwise, we must ensure that the insertion and merging processes are sufficiently *fair* (in the sense of ‘fair’ as used in concurrent programming): we must have some strategy to ensure every possible term that could be inserted *does* get inserted eventually, or we might get stuck inserting other terms infinitely while nonetheless not making progress on the goal.

In some sense, this is to be expected, as even for just equational theorem proving, there are theories which are known to be undecidable. However, even when the process of E-matching, inserting and merging doesn’t terminate, we can consider the infinite fixed-point of this process as the ‘semantics’ of our E-graph [6], and this will be what we will preserve in the shift to EMTs.

2.2.2 Aside: variations on equality saturation

We can do several matches/insertions/merges in between rebuilding; it is sufficient that rebuilding is *eventually* run after changes to the E-graph [8].

There are plenty of extensions to the outer loop that include things such as conditional reasoning (along the lines of: if $a = b$, then assert that $c = d$ by merging classes) or E-class analyses ([8]).

Chapter 3

Egraphs modulo AC

One common pair of identities we want to work with is associativity

$$\forall xyz. f(x, f(y, z)) = f(f(x, y), z) \quad (3.1)$$

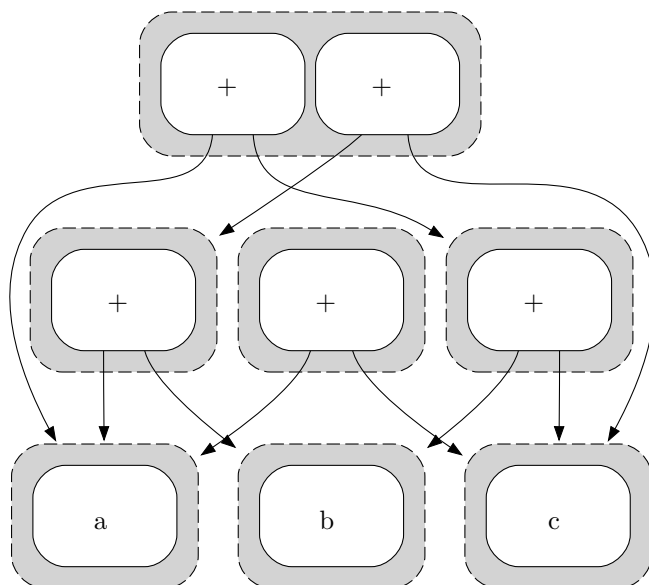
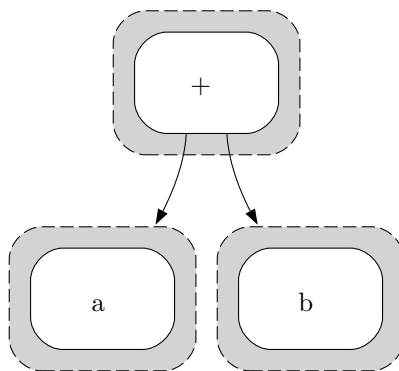
and commutativity

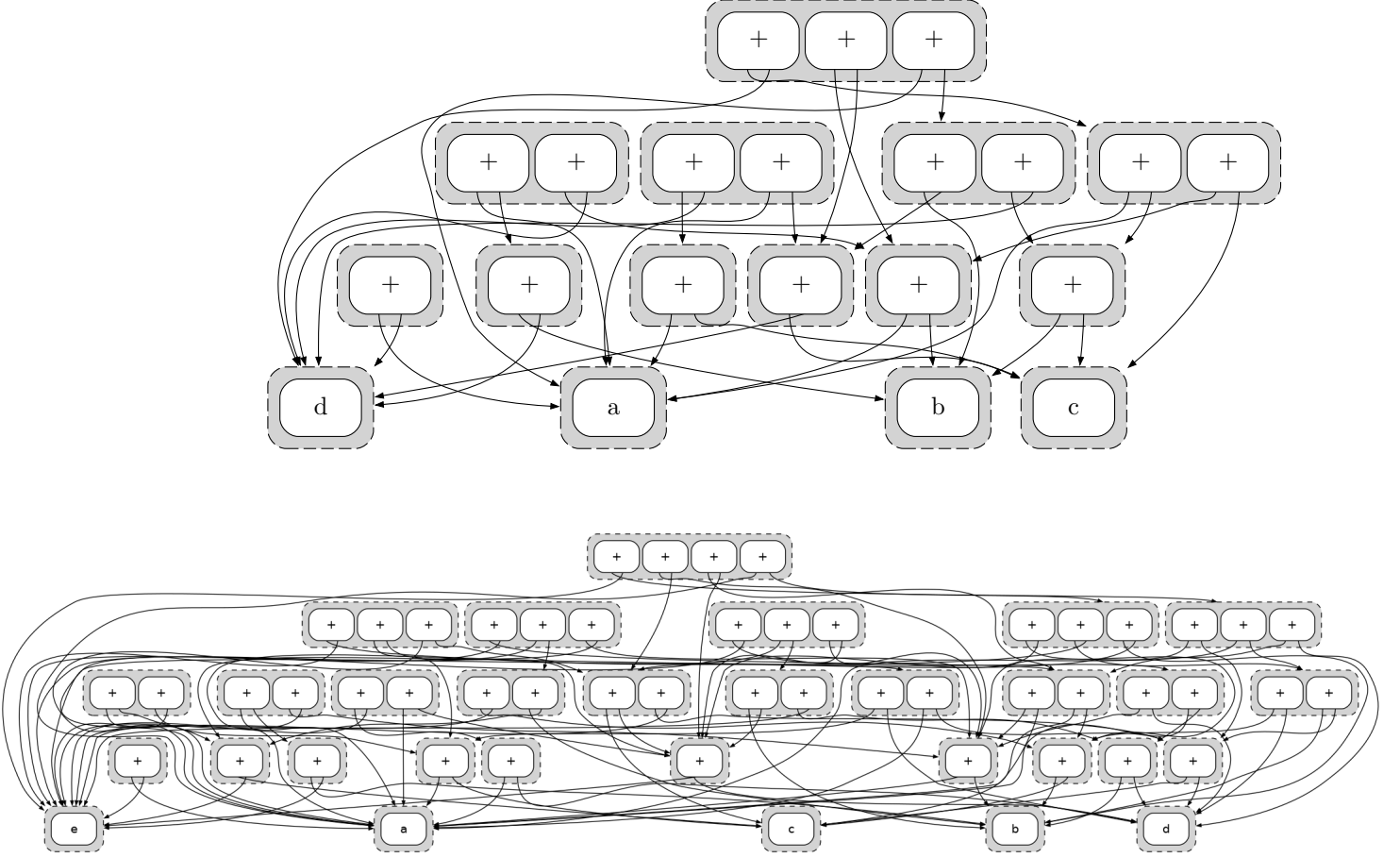
$$\forall xy. f(x, y) = f(y, x) \quad (3.2)$$

We refer to this combination as AC

A common AC operator is $+$, and we will use that as a generic name for an AC operator.

Unfortunately, equality saturation handles these identities rather poorly. Consider as a simple case the terms of length n over some constant terms k_n $t_n = a + (b + (...k_n))$. Then we can see that there is a significant blowup in the size of the saturated graph as we increase n :





It's relatively easy to see why: the top E-class has $2^n - 1$ E-nodes, one for each way to partition the set of atoms into 2 nonempty parts, so the E-nodes in the top class already have $2^n - 1$ E-classes as children. And then each child E-class representing the sum of k atoms has its own 2^k enodes, etc. So, there are $O(2^n)$ E-nodes for each of $O(2^n)$ E-classes, giving a doubly-exponentially $O(2^{2^n})$ sized E-graph.

3.1 Handling AC with EMTs: a first example

The difficult with plain E-graphs motivates developing a notion of *E-graphs modulo theories*, where we work modulo AC (the theory we will focus on now)

We will use an example problem: besides the AC operator $+$, we introduce a function symbol f and the identity $\forall xy. f(x+y)+1 = f(x)+f(y)$. We also allow integer constants k_i , and treat integer arithmetic as an identity: $\forall xy. k_i + k_j =$

k_{i+j} . Indeed, we will write integer constants directly.

We begin with the term $f(a) + (f(b) + f(c))$. We would like to discover that this input term is equal to $2 + f(a + (b + c))$, using the reasoning

$$\begin{aligned}
& f(a) + (f(b) + f(c)) = \\
& f(a) + (f(b + c) + 1) = \\
& (f(a) + f(b + c)) + 1 = \\
& (f(a + (b + c)) + 1) + 1 = \\
& f(a + (b + c)) + (1 + 1) = \\
& f(a + (b + c)) + 2 = \\
& 2 + f(a + (b + c))
\end{aligned}$$

We will present a pictorial overview of how we might solve the above problem using equality saturation on EMTs rather than E-graphs.

The natural first step is to simply not add any different representations of terms that are equal up to AC; thus, nodes in the E-graph for AC operators will be variadic and we will consider their children unordered. This is the key idea from prior work ([9], [3], [4]).

Figure 1 begins with the variadic AC-term $f(a) + f(b) + f(c)$. We omit the labels of equivalence classes.

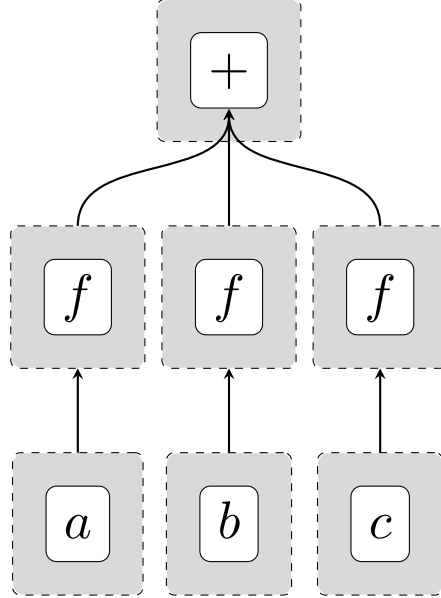
Now, in order to *match* the left-hand side of $f(x) + f(y) = f(x + y) + 1$, we need to do *matching modulo theories*, which will tell us of six matches: $(x, y) \in \{(a, b), (a, c), (b, c), (b, a), (c, a), (c, b)\}$. Then we will add the six substituted forms of $f(x + y) = 1$. Here, we will want to *insert modulo theories*, in a way that keeps the E-graph from representing multiple AC-equivalent terms. The arrows in Figure 2 have become crowded, but top E-class now represents the original term, and $f(a + b) + f(c) + 1$ and its variants with a, b, c permuted.

Now, we once again try to match $f(x) + f(y)$. The only new matches will be $(x, y) = (a + b, c)$ and its permutations, and, modulo AC, the substituted-for term $f(x + y) + 1$ will be the same for all of these. So, we just need to add a single node.

Our last step is to notice that the term $f(a + b + c) + 1 + 1$ has, up to AC, the implicit subterm $1 + 1$, which we can reduce to 2. Having reduced the implicit subterm, we no longer need the 3-part term $f(a + b + c) + 1 + 1$, and can entirely replace it with $f(a + b + c) + 2$.

The final EMT (Figure 3) is saturated, and has ‘discovered’ our desired equality.

Figure 1: EMT of the term $f(a) + f(b) + f(c)$.



FiXme: Should I unfloat the preceding diagrams?

3.2 Challenges for EMT(AC)

We have seen the promising aspects of EMTs for AC, but there is a significant gap in the description given so far, and we need something extra. Consider the equations

$$a + b = b \tag{3.3}$$

$$a + a = c \tag{3.4}$$

$$c + b = d \tag{3.5}$$

taken modulo associativity of $+$.

We can represent them with an E-graph as follows:

Figure 2: EMT of the term $f(a) + f(b) + f(c)$ after some steps.

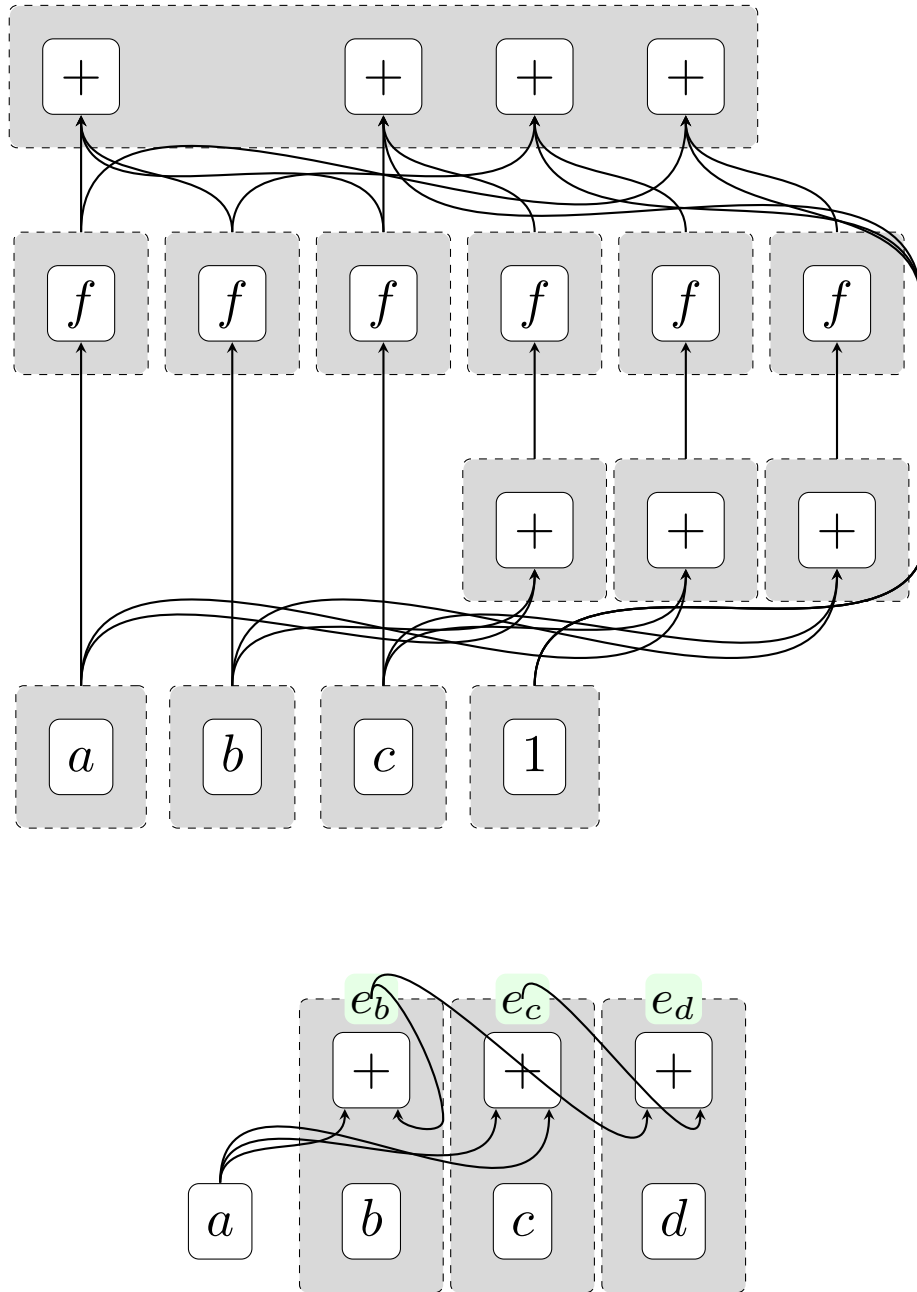
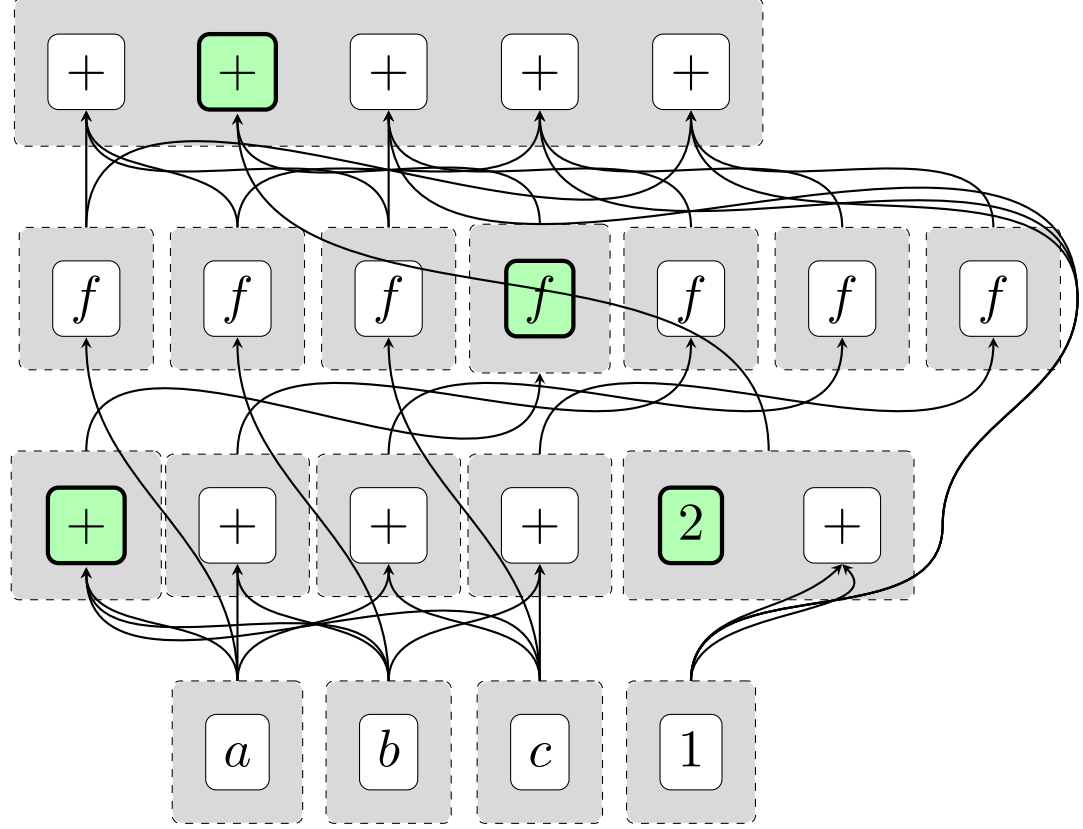


Figure 3: EMT of the term $f(a) + f(b) + f(c)$ once saturated, highlighting the desired term.



Consider the equation $b = d$, which follows from the simple deduction

$$b = a + b = a + (a + b) = (a + a) + b = c + b = d$$

However, e_b and e_d are two distinct E-classes in the E-graph! Why?

The central step in the deduction relies on two different associations of the term $a + a + b$. The association $(a + a) + b$ is correctly assigned to class e_d , while the association $a + (a + b)$ is correctly assigned to the class e_b . However, given that we want to work modulo theories, we need to discover the fact that $e_b = e_d$, or we are throwing out some of the consequences derivable from the theory.

3.3 Filling the gap

To fill the gap, we need to reason about when such an ‘overlapping’ term could occur, while not generating all AC-subterms like equality saturation would.

Our criteria is this: if the EMT contains two terms $t_1 = r + s$ and $t_2 = u + s$ then we need to insert the equation $t_1 + u = t_2 + r$.

This equation is a consequence, since $t_1 + u = (r + s) + u = (u + s) + r = t_2 + r$.

With our above example, we see that e_b contains the E-node $+(a, e_b)$ and e_c contains the E-node $+(a, a)$. Consequently, we insert the equation $e_b + a = e_c + e_b$. Since the right-hand-side is already assigned to e_d and the left-hand-side to e_b , this equation simply asks us to equate e_b and e_d , exactly as desired.

Let’s refine the idea into something more workable. Suppose we have the variadic $+$ -term $+(e_1, \dots, e_k)$ in some E-class e , and the variadic $+$ -term $+(d_1, \dots, d_i)$ in some E-class d . Think of the arguments as multisets, so that $\{e_1, \dots, e_k\} = E$ and similarly for D . Then let the multiset O contain each element with the minimum multiplicity it occurs in either E or D . So if $E = \{e_1, e_1, e_2, e_3\}$ and $D = \{e_1, e_3, e_4\}$, then $O = \{e_1, e_3\}$. If the overlap is empty, we can ignore it. Otherwise, construct the terms $E - O + D$ and $D - O + E$, in the obvious way, and then insert the equation $E - O + D = D - O + E$.

Is this enough? Suppose we imagine a chain of reasoning involving ground equations, the A identity, and the C identity. We can group the reasoning into ‘ground’ sections and the ‘AC’ sections. An AC section, in total, involves reassociating and commuting some term involving $+$ s. So, **FiXme: Write this when I’m less impatient**

3.4 Prospects for EMT(AC)

EMT(AC) avoids the need for equality saturation on the A and C identities by integrating these rules into the underlying E-graph. However, this has a cost: congruence closure normally is polynomial time ¹ while the AC-closure step outlined above is at worst ESPACE ² as will be discussed later.

Moreover, while congruence closure only shrinks the graph by potentially discovering E-classes that can be merged, AC-closure can grow it, so one might want to run AC-closure only intermittently while running congruence closure much more often. This is then not that different in spirit to existing approaches to handling AC blowup in equality saturation by ‘deprioritizing’ the AC identities **FiXme: Do I need a source here**

¹indeed, at best slightly superlinear

²space proportional to $2^{O(n)}$, where n is the size of the E-graph

However, there are two key advantages:

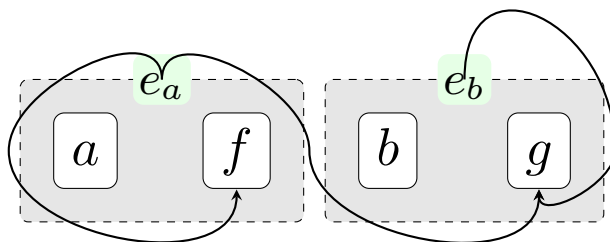
- In many practical cases, the AC-closure is small, while ordinary equality saturation will have to run to completion to be ‘sure’ that nothing is missed.
- AC-closure is much better at finding ‘interesting’ nodes to add to the E-graph, in the sense that the added nodes are part of some nontrivial equality $t_1 + u = t_2 + r$. Equality saturation on AC will generate many ‘trivial’ identities alongside such potentially useful ones.

Chapter 4

E-graphs, EMTs, and rewriting

4.1 E-graphs present a rewriting system

Consider the E-graph



We can read off a set of equations from our final E-graph:

$$\begin{aligned}a &= e_a \\ f(e_a) &= e_a \\ b &= e_b \\ g(e_a, e_b) &= e_b\end{aligned}$$

Given this resulting set of equations, we can determine if any two terms at all are equal under our initial set of equations: simply reduce the terms by treating these equations as a *rewrite system*, read from left-to-right, and see if the normal forms are the same. This is the perspective of Abstract Congruence Closure ([1]): E-graphs present a rewriting system for terms over an *extended signature*. The signature is extended in the sense that we don't just have the symbols f , a , g , etc, but also $e_a, e_b, \dots$. In general, we introduce a new E-class e_i whenever we need to represent a subterm.

Then the congruence closure algorithm, aka E-graph rebuilding, is a matter of restoring *confluence* to this rewrite system. Confluence is the property that any maximal sequence of rewrites end up with the same result. For instance, consider

$$\begin{aligned} f(e_1, e_2) &\rightarrow e_3 \\ f(e_1, e_2) &\rightarrow e_4 \end{aligned}$$

$f(e_1, e_2)$ can rewrite to both e_3 and e_4 . This would violate confluence, unless we have the rewrite $e_3 \rightarrow e_4$ or $e_4 \rightarrow e_3$. Let's say that $e_3 \rightarrow e_4$. Then $f(e_1, e_2) \rightarrow e_3 \rightarrow e_4$, so the divergence $e_3 \leftarrow f(e_1, e_2) \rightarrow e_4$ is only temporary: the left rewrite is not maximal and can be extended.

Of course, confluence applied in this case is simply an instance of the congruence rule: $f(e_1, e_2)$ is in two E-classes that need to be merged by taking the union of e_3 and e_4 , which introduces the rule $e_3 \rightarrow e_4$ or its converse.

4.2 E-graphs simplify confluence

E-graphs in the rewriting perspective consist of two kinds of rules: union-find rules $e_i \rightarrow e_j$ and E-node rules $f(\dots) \rightarrow e_k$. To turn an equation $s = t$ into rules, we flatten it completely into E-node rules, and add a union-find rule when the resulting top E-nodes would be in different classes.

The advantages of this flattening are twofold. Firstly, termination¹ of the rewriting system is easy to determine: E-node rules make a term smaller, so we just need to ensure there is no infinite cycle among the union-find rules. Secondly, confluence is much easier to check for and restore.

Termination and confluence together give convergence, which means that we can apply the rules in any order to a term and get a unique normal form. If the normal form is e_i , then that term is in the E-graph and equivalent to any other

¹in some sectors of computer science, termination is more commonly known as *strong normalization*

term that also yields e_i . We could also get, say, $f(e_1, e_2)$, which means that the term is not in the E-graph, but its two subterms are.

We will use $\rightarrow^*$ for the reflexive transitive closure of the rewriting relation.

Here are all the ways in which an E-graph-like rewriting system could fail to be confluent:

1. $e_1 \rightarrow^* e_2$ and $e_1 \rightarrow^* e_3$
2. $f(e_1, e_2, \dots) \rightarrow^* e$ and $f(d_1, d_2, \dots) \rightarrow^* d$, where for each i , $e_i \rightarrow^* d_i$

That's it! The first kind of case is resolved by the union-find. The second case is precisely congruence.

If, whenever we have the rule $e_i \rightarrow e_j$, we replace e_i everywhere in our rewriting system by e_j , then the confluence rule is even simpler, as we only need to check for $f(e_1, \dots) \rightarrow e$, $f(d_1, \dots) \rightarrow d$, and $e_i = d_i$ or $e_i \rightarrow d_i$. Now we don't even have to think about $\rightarrow^*$.

So, E-graphs simplify the problem of congruence closure by reducing the problem to checking for certain kinds of non-closure (local failures of confluence), resolving them, and repeating until fixpoint. In this way there is no complexity to handling deeply-nested subterms.

Such local failures of confluence are known as critical pairs².

4.3 Equality saturation: handling AC-critical pairs by brute force

When we want to reason about AC, we need the notion of AC-critical pair. If $e_1 + e_2 \rightarrow e_3$ and $e_4 + e_2 \rightarrow e_5$, then there is a critical term $e_1 + e_2 + e_4$, which is AC-equal to $(e_1 + e_2) + e_4$ which rewrites to $e_3 + e_4$, and also to $e_1 + (e_2 + e_4)$ which rewrites to $e_1 + e_5$. So, our critical pair is $e_3 + e_4$ and $e_1 + e_5$.

E-graphs handle AC-critical pairs the same way as critical pairs for any other theory, with equality saturation: we *ground* the AC identities by matching either side with the E-graph, and adding the equivalent other side (if it is not present) and merging the equivalence classes corresponding to either side.

Why do E-graphs fare poorly with AC? We have seen that equality saturation with AC will generate a doubly-exponential number of E-nodes.

These 'redundant' E-nodes and E-classes play a role: any of the $O(2^{2^n})$ AC-subterms of the input could be part of an AC-critical pair, and so equality

²They are also required to be minimal, but we gloss over this here

saturation eagerly assigns new E-classes to them, just in case. Introducing these extra E-classes and E-nodes is seemingly the best we can do in general unless we have a clever way to detect which ground instances of the identities are needed in order to ensure confluence modulo those identities.

4.4 EMTs: Handling AC-critical pairs less naïvely

We don't need to add all AC-subterms to find critical pairs. It suffices to consider every pair of E-nodes with an AC symbol and check them for overlap.

This procedure always terminates ([1]). It is, in fact, a special case of Buchberger's algorithm for finding Gröbner bases of sets of polynomials ([7]), a well-studied algorithm with a lot of work done in optimizing it to be fast enough in practice. **FiXme: I think we will have proved termination in the introductory section by this point**

So, if an E-graph has E-node rules and union-find rules, EMT(AC) adds a third kind, an AC-rule.

For AC-rules, do we want to allow rules of the form $e_1 + \dots + e_n \rightarrow d_1 + \dots + d_k$ or restrict to simply $e_1 + \dots + e_n \rightarrow e$? The former is closer to usual presentations of Buchberger's algorithm, while the latter is more in the spirit of E-node rules. We will refer to these as the *side-table* and *integrated* approaches. Both are valid, but there is a subtlety in the integrated approach that might be missed, while the side-table approach makes it much more obvious.

4.4.1 EMT(AC)s via a side-table

With the side-table approach, we think of the set of AC rules $e_1 + \dots + e_n \rightarrow d_1 + \dots + d_n$ as being a separate thing entirely to the E-graph.

On insertion, we don't add any E-node rules whenever the function symbol is an AC symbol $+$, but generate a new E-class name e and add the AC-rule $e_1 + \dots + e_i \rightarrow e$ when we insert the flattened term $e_1 + \dots + e_i$.

To compute the AC-closure, we take every pair of two AC-rules $r \rightarrow s$ and $t \rightarrow u$. Consider r, s, t, u to be multisets, and define $+$, $-$, $\max$, $\min$ as operations lifted across them, acting on the multiplicities. Then we can define

- The overlap: $\min(r, t)$
- The critical term: $\max(r, t)$
- The critical pair: $c_1 = \max(r, t) - r + s$ and $c_2 = \max(r, t) - t + u$

Note that $\max(r, t) + \min(r, t) = r + t$, so it is also possible to write the critical pair as $c_1 = t + s - \min(r, t)$ and $c_2 = r + u - \min(r, t)$.

When the overlap is not 0, then we take the critical pair and try to rewrite both sides by existing rules until it is minimal. If it is still non-trivial, we *orient* it, according to some ordering on flat AC-terms. A common choice for Buchberger’s algorithm is *grevlex* order, graded reverse-lexicographic, which is used in our implementation.

After orienting the reduced rule, we reduce every other rule with respect to it, and add it to the set of AC rules.

FiXme: This section needs a bit more detail/clarity, probably

The final ingredient is combination rules. If we were ever to have a reduced rule $e_1 \rightarrow e_2$, with 1-term sums on either side, we should instead treat this as a union-find rule, and correspondingly propagate its consequences into the union-find and E-node rules. And vice versa, whenever the union-find rules are updated, we need to update any AC-rules according to the new ordering on atoms induced by the union-find rule.

4.4.2 Integrated EMT(AC)s

Unlike the side-table approach, here we will only allow AC rules with an atomic right-hand-side, thus making them of the same form as E-node rules.

As before, we can now form critical pairs c_1, c_2 for rules with non-empty overlap. We proceed as above and reduce the critical terms as much as possible by existing rules. Then we need to create a new E-class id e and insert the rules $c_{1\text{reduced}} \rightarrow e$ and $c_{2\text{reduced}} \rightarrow e$.

One subtlety is that here we want to reduce *existing* rules by the new one. This interreduction step is a standard step in Buchberger’s algorithm, but may not be obvious in the ‘integrated’ viewpoint, as it is a form of destructive rewriting on the E-graph.

This interreduction shrinks the size of the E-graph, so it is important for performance.

FiXme: Can we prove these approaches equivalent easily?

Chapter 5

Beyond rewriting: E-matching and E-analysis

Chapter 6

Implementation and evaluation

6.1 Laziness and incrementality

6.1.1 Union-find laziness

For congruence closure, and thus E-graphs, the laziness possible in the union-find seems difficult to make use of, as we need to detect the congruence $f(a_1, \dots, b, \dots, a_n) = f(a_1, \dots, b, \dots, a_n)$ once b and c have been equated; it is not enough to just ‘discover’ that $b = c$ when calling find on b or c or one of their equivalents, as such a find call is not guaranteed to happen at any particular point before we need to discover the congruence.

6.2 Evaluation

Chapter 7

EMTs for other theories

Chapter 8

Prior work

FiXme: It feels better to include this stuff in the main EMT section? The ‘canonizer’ and ‘containers’ approach is what prior work consists of, and tacking it on here feels a bit strange. On the other hand, I guess I need somewhere to cite Bachmair, Kapur, Tiwari, and Conchon

FiXme: Actually, I might put it here...

Chapter 9

Conclusion and future work

FiXme: EMTs for other theories. Buchberger extended to polynomials etc.
F4/F5 algorithms. Hermite/Gauss elimination

Bibliography

- [1] Leo Bachmair and Ashish Tiwari. “Abstract Congruence Closure and Specializations”. In: *Automated Deduction - CADE-17*. Ed. by David McAllester. Berlin, Heidelberg: Springer, 2000, pp. 64–78. ISBN: 978-3-540-45101-3. DOI: [10.1007/10721959_5](https://doi.org/10.1007/10721959_5).
- [2] Ernst W Mayr and Albert R Meyer. “The complexity of the word problems for commutative semigroups and polynomial ideals”. In: *Advances in Mathematics* 46.3 (Dec. 1, 1982), pp. 305–329. ISSN: 0001-8708. DOI: [10.1016/0001-8708\(82\)90048-2](https://doi.org/10.1016/0001-8708(82)90048-2). URL: <https://www.sciencedirect.com/science/article/pii/0001870882900482> (visited on 03/11/2026).
- [3] Pavel Panchekha. *LIN-graphs, an Egraph modulo T*. Pavel’s Blog. May 23, 2025. URL: <https://pavpanchekha.com/blog/lin-graphs.html>.
- [4] Saul Shanabrook. *Custom Data Structures in E-Graphs*. PLSE Blog. Feb. 24, 2026. URL: <https://uwplse.org/2026/02/24/egglog-containers.html>.
- [5] Natarajan Shankar. “Little Engines of Proof”. In: *FME 2002: Formal Methods—Getting IT Right*. Ed. by Lars-Henrik Eriksson and Peter Alexander Lindsay. Red. by Gerhard Goos, Juris Hartmanis, and Jan Van Leeuwen. Vol. 2391. Series Title: Lecture Notes in Computer Science. Berlin, Heidelberg: Springer Berlin Heidelberg, 2002, pp. 1–20. ISBN: 978-3-540-43928-8. DOI: [10.1007/3-540-45614-7_1](https://doi.org/10.1007/3-540-45614-7_1). URL: http://link.springer.com/10.1007/3-540-45614-7_1 (visited on 03/17/2026).
- [6] Dan Suciu, Yisu Remy Wang, and Yihong Zhang. *Semantic foundations of equality saturation*. Jan. 5, 2025. DOI: [10.48550/arXiv.2501.02413](https://doi.org/10.48550/arXiv.2501.02413). arXiv: [2501.02413\[cs\]](https://arxiv.org/abs/2501.02413). URL: <http://arxiv.org/abs/2501.02413> (visited on 02/04/2026).
- [7] Ashish Tiwari. *Decision procedures in automated deduction*. State University of New York at Stony Brook, 2000. URL: <https://search.proquest.com/openview/30046982652b13cad7b85d653cdf547/1?pq-origsite=gscholar&cbl=18750&diss=y> (visited on 03/18/2026).
- [8] Max Willsey et al. “egg: Fast and extensible equality saturation”. In: *Proceedings of the ACM on Programming Languages* 5 (POPL Jan. 4, 2021), pp. 1–29. ISSN: 2475-1421. DOI: [10.1145/3434304](https://doi.org/10.1145/3434304). URL: <https://dl.acm.org/doi/10.1145/3434304> (visited on 02/08/2026).

- [9] Philip Zucker. *Omelets Need Onions: E-graphs Modulo Theories via Bottom-up E-matching*. Apr. 19, 2025. DOI: [10.48550/arXiv.2504.14340](https://doi.org/10.48550/arXiv.2504.14340). arXiv: [2504.14340\[cs\]](https://arxiv.org/abs/2504.14340). URL: <http://arxiv.org/abs/2504.14340> (visited on 01/20/2026).